Code-Layout, Readability and Reusability

Clean architecture assessment

Project: AI WeatherWise API   Track: AI-ML & GEN-AI Track

Prepared: 07 October 2026   Team ID: Not provided

Maximum Marks: 5

Team Members & Assigned Roles

Team Member	Role
Jai Bharath A	Team Lead & Lead Backend/API Architect
Mukesh Balaji	Generative AI & Prompt Engineering Engineer
Yadagiri Mani	Database & Data Engineering Specialist
Deepak G	Security, Testing & Reliability Engineer
Naveen	API Integration, Documentation & Demonstration Engineer

Quality	Expected practice	WeatherWise implementation
Structure	Logical module separation	config / controllers / middleware / models / routes / services
Naming	Descriptive camelCase names	weatherService, aiService, authMiddleware, etc.
Reusability	Centralize repeated logic	Token helper, weather service, AI service, error handling
Security	No secret hard-coding	Environment variables
Error handling	Consistent JSON errors	Centralized Express error handler
Database	Schema/index definitions near models	Mongoose models and compound index
Async code	Use async/await with controlled failures	External API and database operations

Reusable Components

Component	Reused in	Value
authMiddleware.js	Profile + locations routes	Single JWT authorization guard
weatherService.js	Weather controller and AI context preparation	One weather retrieval/fallback path
aiService.js	Summary + recommendation endpoints	Centralized Gen-AI client and fallback rules
Token helper	Registration/login	Consistent signed token generation
Error handler	All controllers	Standardized error responses

Readability Checklist

- Keep controllers thin; business logic belongs in services.
- Validate inputs before database writes or external calls.
- Use constants for repeated status messages and configuration.
- Comment only where reasoning is non-obvious; avoid comments that restate code.
- Keep endpoint contracts stable and document breaking changes.